

Assignment Fair Split Bill C++

Emmanuel Timothy Budianto, Athallah Shafi Ramadhan, Alidya Kharisma, Muhammad Frandyka Kurniawan
Faculty of Mathematics and Natural Sciences
Gadjah Mada University
Yogyakarta, Indonesia

muhammadfrandykakurniawan@mail.ugm.ac.id, athallahshafiradamadhan@mail.ugm.ac.id,
alidyakharisma.a@mail.ugm.ac.id, emmanueltimothybudianto@mail.ugm.ac.id

Abstract—University students and friends often share meals, but calculating each person's payment can be difficult when they consume different items or portions. Dividing the total bill equally may be unfair, especially when some people order more than others. Therefore, this mini project aims to develop a Fair Split Bill System using C++. The program asks users to enter the participants, item names, item prices, and each person's portion of every item, including fractional portions for shared items. Based on this information, the system calculates each person's subtotal using a portion-based formula, applies a tax percentage proportionally to each subtotal, and displays an itemized receipt with the final amount each person must pay. The program uses structures, arrays, loops, and conditional statements covered in our programming course. This project helps reduce manual calculation errors when splitting shared expenses. However, the current version has limitations, such as the absence of file storage, input validation, and a graphical user interface. Future development may include file storage, input validation, service-charge and discount features, and payment tracking.

Keywords—*split bill system, shared expenses, portion-based calculation, C++, tax calculation, expense management*

I. INTRODUCTION

Sharing expenses is common in everyday life. Students and friends often eat together, travel together, or purchase items as a group. After making a payment, they need to determine how much each person should contribute. This process becomes difficult when different people order different items or consume different portions. If the total bill is divided equally, some people may pay more than they should, while others may pay less. Previous research also found that diners who split the bill

equally tend to order more than those who pay individually [1].

Manual bill calculation can also cause mistakes. Users may forget an item, enter the wrong price, calculate the portions incorrectly, or fail to include tax. These problems become more complicated when there are many participants and several items in one bill. Moreover, many existing bill-splitting applications still rely on basic equal splits, which are not suitable for shared meals where each person consumes a different portion [2]. Therefore, a system that can calculate each person's payment automatically is useful for improving accuracy and fairness.

To address this problem, we developed a C++ program called the Fair Split Bill System. The program allows users to enter the names of the participants, the items purchased, the price of each item, and the portion consumed by each person. The system then calculates the individual share for every item using the following formula:

$$\text{Person's Item Share} = \text{Item Price} \times \frac{\text{Person's Portion}}{\text{Total Portions}}$$

After calculating the share for every item, the program adds the results to obtain each participant's subtotal. Then, the system calculates the tax amount from the tax percentage entered by the user and distributes it proportionally based on each participant's subtotal. The final output shows a clear amount that every participant must pay. The objective of this project is to provide a simple and accurate tool for dividing shared meal costs based on actual consumption.

II. METHODOLOGY

A. Portion-Based Approach

The program divides a bill according to what each person actually consumed. Every item has a price per portion, and the user enters how much of that item each person ate as two numbers: the portion eaten and the total portions. A shared item is therefore divided by fraction. For example, if a person ate 3 of 9 equal parts of an item, the quantity is 3/9. A whole portion is entered as "1 1" and no consumption as "0 1".

B. Calculation Formulas

The calculation has three steps. First, the quantity consumed and the share of each item are calculated for every person:

$$\text{Quantity} = \text{Portion Eaten} / \text{Total Portions}$$

$$\text{Item Share} = \text{Item Price} \times \text{Quantity}$$

Second, the shares of all items are added to obtain the person's subtotal:

$$\text{Subtotal} = \text{sum of Item Share}$$

Third, the tax is calculated from the tax percentage entered by the user and distributed according to each person's share of the total subtotal:

$$\text{Total Tax} = \text{Total Subtotal} \times (\text{Tax Percentage} / 100)$$

$$\text{Person's Tax} = (\text{Person's Subtotal} / \text{Total Subtotal}) \times \text{Total Tax}$$

$$\text{Total to Pay} = \text{Person's Subtotal} + \text{Person's Tax}$$

If there is no tax, the percentage is entered as 0. A person who consumes more therefore pays more for both the items and the tax.

C. Program Flow

Figure 1 shows the flow of the program. The program starts by reading the number of people and their names, then the number of items with their names and prices. These two inputs are handled by simple loops. After that, a nested loop asks for the portion eaten and the total portions for every item of every person, and the result is added to that person's subtotal. When all portions are entered, the program reads the tax percentage, calculates each person's tax and total, and prints the bill for every person together with the grand totals.

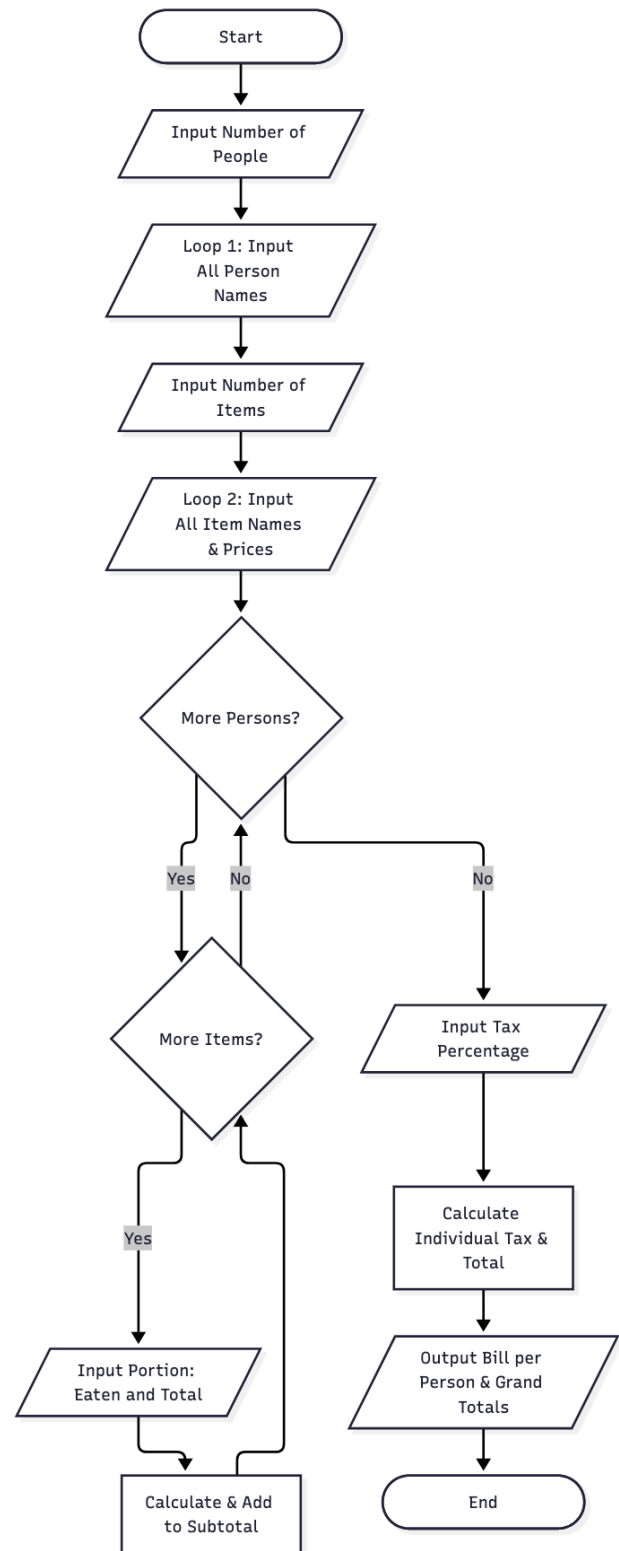


Fig. 1. Flowchart of the Fair Split Bill System

III. IMPLEMENTATION

The program is written as a single C++ source file that uses the `iostream`, `string`, and `iomanip` libraries. The `iostream` library handles keyboard input and screen output, `string` stores participant and item names, and `iomanip` formats the amounts shown in the receipt. All steps run in `main()` in sequence: input, calculation, and output. Two constants, `MAX_PEOPLE` and `MAX_ITEMS`, both set to 100, define the size of the arrays.

A. Data Structures

Two structures store the data, as shown in Listing 1. The `Item` structure holds the name and the price per portion of an item. The `Person` structure holds the participant's name, an array `consumption[]` with the portion consumed of every item, and the subtotal. Up to 100 participants and 100 items are kept in arrays of these structures. The consumption array is also reused later to print the itemized receipt.

```
struct Item {
    string name;
    double price;
};

struct Person {
    string name;
    double consumption[MAX_ITEMS];
    double subtotal;
};
```

Listing 1. Structures used in the program

B. Input Process

The program first asks for the number of participants and their names, then the number of items with the name and price per portion of each item. The item list is entered only once. After that, the program asks for the portion consumed by each participant for every item. A portion is entered as two numbers, eaten and total portions. For example, "1 1" means one whole portion, "0 1" means nothing was consumed, and "6 9" means six-ninths of the item. This allows shared items to be divided according to what each person actually ate. Because names are read with `cin`, each name must be a single word without spaces.

C. Portion-Based Calculation

For each participant and item, the quantity consumed is calculated as eaten divided by total portions, as shown in Listing 2. The quantity is saved in the consumption array and multiplied by the item price, and the result is added to the participant's subtotal. A conditional expression prevents division by zero when the total portion is zero.

```
double qty = (totalPortions > 0) ? (eaten /
totalPortions) : 0;

people[i].consumption[j] = qty;
people[i].subtotal += qty * items[j].price;
```

Listing 2. Portion-based subtotal calculation

D. Tax Distribution

After all consumption is entered, the user inputs the tax percentage (0 if there is no tax). The program adds all participant subtotals to get the grand subtotal, calculates the total tax from the percentage, and distributes it according to each participant's share of the grand subtotal, as shown in Listing 3. A participant who consumed more therefore pays a proportionally larger part of the tax.

```
double totalTax = totalSubtotal * (taxPercent
/ 100.0);

double personTax = (totalSubtotal > 0)
    ? (people[i].subtotal / totalSubtotal) *
totalTax : 0;
```

Listing 3. Proportional tax calculation

E. Program Output

Figure 2 shows an example run with three participants (Alice, Bob, and Charlie), three items, and a tax of 11%. After the inputs are entered, the program prints a receipt for each participant, followed by the grand totals.

```

Enter number of people: 3
Person 1 name: Alice
Person 2 name: Bob
Person 3 name: Charlie

Enter number of items: 3
Item 1 name & price: Chicken_Bucket 180000
Item 2 name & price: Nasi_Goreng 25000
Item 3 name & price: Es_Teh 5000

--- Enter Portion Eaten (e.g. '6 9' for 6/9, '1 1' for whole, '0 1' for none) ---

Consumption for Alice:
Chicken_Bucket (Eaten / Total): 6 9
Nasi_Goreng (Eaten / Total): 0 1
Es_Teh (Eaten / Total): 2 1

Consumption for Bob:
Chicken_Bucket (Eaten / Total): 3 9
Nasi_Goreng (Eaten / Total): 1 1
Es_Teh (Eaten / Total): 0 1

Consumption for Charlie:
Chicken_Bucket (Eaten / Total): 0 9
Nasi_Goreng (Eaten / Total): 1 1
Es_Teh (Eaten / Total): 1 1

Enter tax percentage: 11

===== BILL RESULT =====

Person: Alice
Items:
- Chicken_Bucket x0.67 = Rp120000
- Es_Teh x2.00 = Rp10000
Subtotal: Rp130000
Tax: Rp14300
Total: Rp144300

Person: Bob
Items:
- Chicken_Bucket x0.33 = Rp60000
- Nasi_Goreng x1.00 = Rp25000
Subtotal: Rp85000
Tax: Rp9350
Total: Rp94350

Person: Charlie
Items:
- Nasi_Goreng x1.00 = Rp25000
- Es_Teh x1.00 = Rp5000
Subtotal: Rp30000
Tax: Rp3300
Total: Rp33300

=====
Grand Subtotal: Rp245000
Grand Tax: Rp26950
Grand Total: Rp271950

```

Fig. 2. Example programs run with three participants and 11% tax

Each receipt lists only the items the participant consumed, with the subtotal, tax, and total to pay. For example, Alice ate 6/9 of the Chicken_Bucket and drank two Es_Teh, so her subtotal is Rp130.000 and her total is Rp144.300 after Rp14.300 of tax. The tax is shared in proportion to each subtotal, and the grand total of Rp271.950 equals the sum of all individual payments.

IV. RESULT AND DISCUSSION

To evaluate the Fair Split Bill System, a simulated dining scenario was run with three participants (Alice, Bob, and Charlie) sharing a Chicken_Bucket (Rp180.000), Nasi_Goreng (Rp25.000), and Es_Teh (Rp5.000), with an 11% tax. The fractional input made it possible to record different consumption patterns. For instance, Alice ate six out of nine parts (6/9) of the shared Chicken_Bucket and two whole portions (2/1) of Es_Teh, but did not eat any Nasi_Goreng (0/1).

The program produced a Grand Subtotal of Rp245.000 and a Grand Total of Rp271.950 after the tax. If the bill were split equally, each participant would pay Rp90.650. With the fair split, Alice pays Rp144.300 (Rp53.650 more than the equal amount), including Rp14.300 of tax. Bob pays Rp94.350 (Rp3.700 more). Charlie pays only Rp33.300 (Rp57.350 less), which comes from one portion each of Nasi_Goreng and Es_Teh plus Rp3.300 of tax. The differences add up to zero, so the total paid stays the same. These results show that with an equal split, people who consume less end up subsidizing those who consume more.

The fractional inputs were converted correctly into decimal quantities. For example, Bob's 3/9 portion of the Chicken_Bucket became a quantity of 0.33 (shown as x0.33), which gave exactly Rp60.000 for that item. The total tax (Rp26.950) was calculated once from the Grand Subtotal and distributed in proportion to each personal subtotal, so the three individual totals add up exactly to the Grand Total. This confirms that the program achieves its objective of replacing arbitrary division with an accurate calculation based on actual consumption. One limitation observed during testing is that names must be a single word and the inputs are not validated, so invalid input may cause errors.

V. CONCLUSION AND FUTURE WORK

We developed the Fair Split Bill System because traditional equal bill-splitting is inherently unfair, often causing people who eat less to pay more than they should while shared items are not divided accurately. The program records the participants, the items with their prices, and the portion consumed by each person, and divides shared items strictly by the consumed portion. It turns these inputs into a personal subtotal and distributes the applied tax proportionally. We believe this meets the objective of the project by replacing arbitrary division with an accurate, consumption-based calculation. However, it is currently a desktop-oriented C++ program with no input validation, and its manual data entry is not always practical in a real-time restaurant environment.

Several improvements are planned for our future work. The first is to add input validation and a service-charge feature to the current program. The second is to transition the software into a graphical mobile application, making it more accessible at the dining table. The third is to implement Optical Character Recognition (OCR) technology to automatically scan and parse physical receipts, which would eliminate the manual input of every item name and price. The fourth is to integrate direct payment gateway links, allowing users to transfer their exact calculated amounts to the person paying the bill. Lastly, file storage can be added to save bills and receipts for later use, and it can later be expanded into a cloud-based database to track group expense histories over time.

REFERENCES

- [1] U. Gneezy, E. Haruvy, and H. Yafe, "The inefficiency of splitting the bill," *The Economic Journal*, vol. 114, no. 495, pp. 265–280, Apr. 2004.
- [2] B. Shaikh, A. Khan, A. Shaikh, and N. Yadav, "A study on the advancement of bill splitting and expense management applications," *Int. J. Progressive Research in Engineering Management and Science (IJPREAMS)*, vol. 5, no. 4, pp. 2544–2550, Apr. 2025.